Enterprise Agentic RAG Support Automation Platform

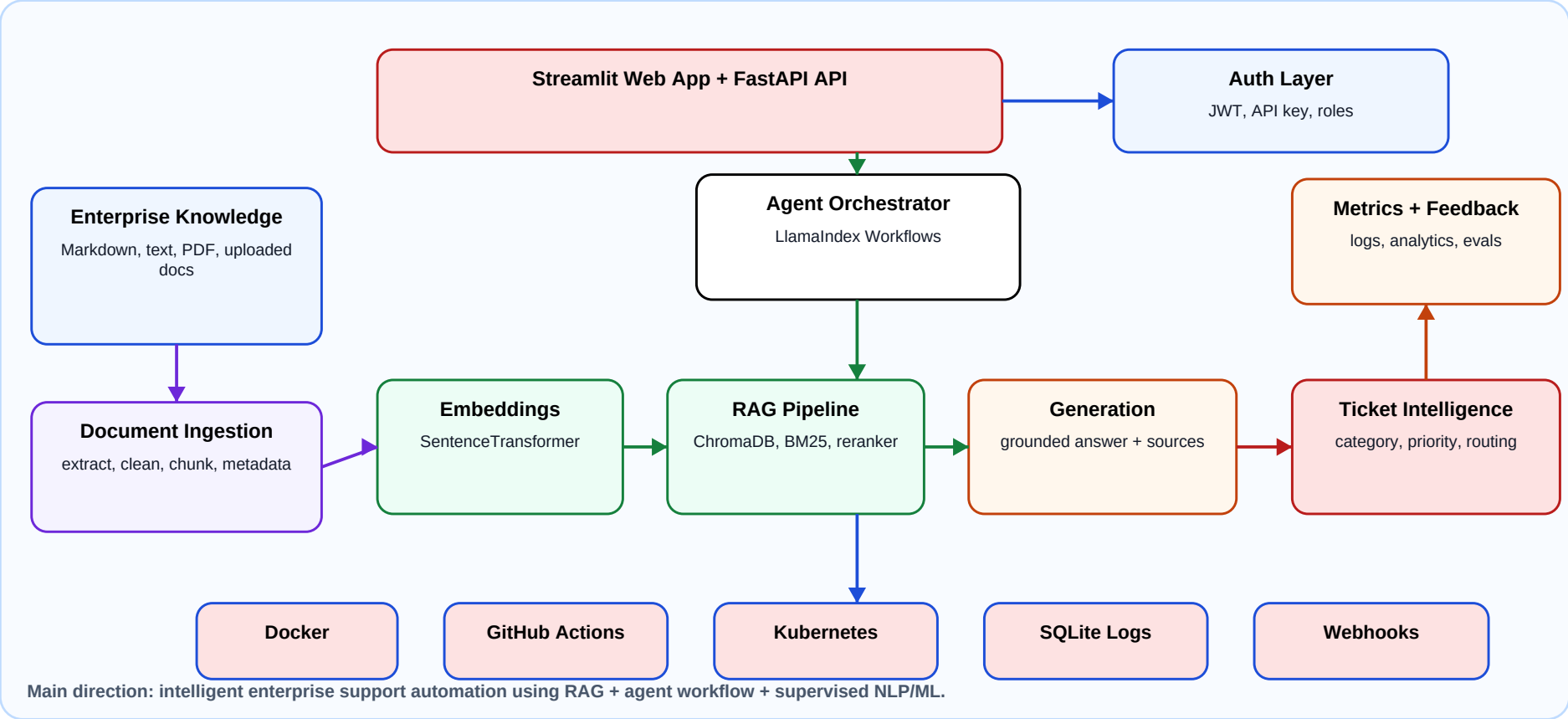
Architecture and flow explainer for interview discussion

A production-oriented AI support automation platform that helps enterprise IT teams answer, triage, route, and respond to support issues using LlamaIndex Workflows orchestration, Retrieval-Augmented Generation, and supervised NLP/ML ticket intelligence.

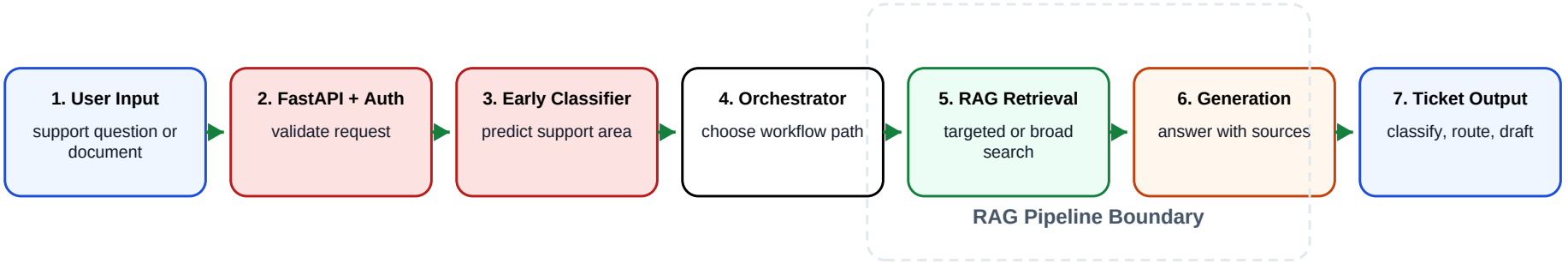
Project Direction	Intelligent enterprise support automation system
Core AI Pattern	RAG with agentic workflow orchestration
ML/NLP Role	Ticket category and priority prediction
Engineering Focus	FastAPI, Streamlit, auth, logs, metrics, Docker, Kubernetes
End Goal	Reduce repeated support effort and improve ticket quality

1. System Architecture

The system is not just a chatbot. It is an intelligent workflow system: the UI/API receives a support issue, the orchestrator coordinates retrieval and ticket decisions, the RAG layer narrows the knowledge base, and the output includes a grounded answer plus ticket-ready metadata.



2. End-to-End Flow



Step	What Happens	Why It Matters
Input	User asks a support question or uploads a document.	Captures the support problem and expands the knowledge base.
Early Classification	A lightweight classifier predicts the likely support area such as VPN, account access, or MFA.	Guides RAG toward the right knowledge-base sources before answer generation.
Orchestration	LlamaIndex Workflows controls pre-classification, retrieval, generation, ticket intelligence, and next action.	Makes the app agentic instead of a single prompt call.
Retrieval	ChromaDB vector search, BM25 keyword search, metadata filtering, and reranking select evidence. If early classification is uncertain, retrieval falls back to broad search.	Narrows the knowledge base while still handling unknown issues safely.
Generation	LangChain/OpenAI can generate grounded answers, with deterministic fallback for offline demos.	Keeps answers source-backed and demo-safe.
Ticket AI	Rules plus supervised NLP/ML predict category, priority, team routing, and ticket draft.	Turns answers into operational support actions.
Observability	Logs, feedback, latency, confidence, and evaluation metrics are captured.	Shows production thinking and measurable quality.

3. Component Responsibilities

Use this page to explain what each part owns. The key interview framing: RAG answers the question, the workflow decides what to do next, and ML/NLP improves ticket intelligence.

Component	Responsibility
Streamlit UI	Ask questions, view answers/sources, upload documents, review analytics.
FastAPI Backend	Exposes REST endpoints, validation, auth, health/readiness, metrics, and ticket APIs.
LlamaIndex Workflows	Coordinates the staged support automation flow and next-action decisions.
RAG Pipeline	Chunks documents, embeds content, retrieves with vector + BM25, filters by domain, reranks.
Grounded Generation	Uses retrieved context to answer with sources; supports LLM mode and offline fallback.
Ticket Intelligence	Classifies category, predicts priority, recommends team, and creates ticket draft.
PyTorch Multi-Task Model	Uses shared layers with category and priority heads for supervised ticket prediction.
Persistence + Logs	Stores query logs, feedback, analytics data, and ticket draft records.
Deployment Layer	Docker, Docker Compose, Kubernetes manifests, ConfigMap, Secrets, PVCs, and CI.

4. Evaluation Metrics

The project measures more than whether the app runs. It measures whether retrieval finds the right source, whether answers stay grounded, whether ticket decisions are correct, and whether the workflow is fast enough for a support tool.

Area	Metrics	Current Result
Retrieval	Retrieval Accuracy, Precision@K, Recall@K, Top-1 Accuracy, MRR, nDCG@K	100%, 97.50%, 100%, 100%, 100%, 100%
Answer Quality	Grounded Answer Rate, Faithfulness Heuristic Rate	100%, 100%
Workflow	Category Accuracy, Priority Accuracy, Team Routing Accuracy, Safe Agent Decision Rate	90%, 100%, 100%, 100%
ML/NLP	PyTorch Category Accuracy, PyTorch Priority Accuracy, Logistic Baseline comparison	100%, 87.50%, baseline tracked
Performance	Average latency, workflow latency, answer-stage latency	53.96 ms, 52.72 ms, 52.36 ms

How to explain the metric story:

- RAG metrics prove the system can find the right internal knowledge before answering.
- Groundedness and faithfulness prove the answer is tied to retrieved evidence.
- Ticket metrics prove the system can turn natural language issues into operational decisions.
- PyTorch vs Logistic Regression comparison proves ML experimentation and baseline thinking.

5. How RAG and Answer Metrics Are Calculated

These metrics are calculated from curated support questions with expected source documents. They explain whether the RAG layer retrieved useful evidence and whether the final answer stayed tied to that evidence.

Metric	How It Is Calculated	Why This Metric Matters
Retrieval Accuracy	Questions where at least one expected source appears in retrieved sources / total questions.	Shows whether RAG can find the right document before generation starts.
Precision@K	Relevant retrieved sources in top K / total retrieved sources in top K.	Prevents the system from returning too many noisy or unrelated citations.
Recall@K	Expected relevant sources found in top K / total expected relevant sources.	Checks whether the system missed any required evidence from the knowledge base.
Top-1 Source Accuracy	Questions where the highest-ranked source is expected / total questions.	Measures whether the first source shown to the LLM/user is the best evidence.
MRR	Average of 1 / rank of first correct source. Rank 1 = 1.0, rank 2 = 0.5.	Rewards systems that place the first correct answer source near the top.
nDCG@K	Ranking score that gives more credit when relevant sources appear near the top.	Evaluates ranking quality when multiple relevant sources may exist.
Grounded Answer Rate	Answers supported by retrieved knowledge chunks / total answers.	Proves the answer is based on internal documents, not generic model memory.
Faithfulness Rate	Answers that avoid unsupported claims using a heuristic check / total answers.	Reduces hallucination risk and checks if generated content stays within evidence.

6. How Workflow and ML Metrics Are Calculated

These metrics are calculated by comparing the agent output against expected ticket labels, support teams, safe decisions, ML labels, confidence behavior, and measured runtime.

Metric	How It Is Calculated	Why This Metric Matters
Category Accuracy	Correct ticket categories / total evaluation questions.	Shows whether the system understands what type of support issue the user has.
Priority Accuracy	Correct ticket priorities / total evaluation questions.	Shows whether the support automation can correctly identify urgency.
Team Routing Accuracy	Correct support-team routing decisions / total evaluation questions.	Measures whether tickets go to the right team instead of causing manual rerouting.
Safe Agent Decision Rate	Safe next-action decisions / total evaluation questions.	Checks whether the agent chooses safe actions like answer, clarify, escalate, or urgent draft.
Weighted F1	Class-balanced F1 score for category or priority labels.	Useful because some labels may appear more often than others in real support data.
PyTorch Accuracy	Correct predictions from the PyTorch multi-task classifier / test examples.	Shows whether the neural model learned ticket category and priority patterns.
Logistic Baseline Accuracy	Correct predictions from TF-IDF + Logistic Regression / test examples.	Provides a simple baseline so PyTorch improvement can be compared honestly.
Average Confidence	Average workflow confidence score across evaluation questions.	Helps decide when the system should answer directly or escalate to a human.
Latency	Average measured execution time across API, workflow, and answer stages.	Shows whether the system is responsive enough for a support workflow.

7. Components Used

This list helps you explain the project by layer. The important idea is that every component has a job: retrieve knowledge, reason over it, turn it into a support action, secure it, persist it, and measure it.

Layer	Components Used
Frontend	Streamlit web app for chat, analytics, document upload, and support workflow demos.
API Backend	FastAPI, Pydantic validation, Swagger/OpenAPI docs, health and readiness endpoints.
Orchestration	LlamaIndex Workflows for staged retrieval, generation, ticket intelligence, confidence, and decisions.
RAG Retrieval	Document chunking, SentenceTransformers embeddings, ChromaDB vector store, BM25 keyword search, metadata/domain filtering, reranking.
Generation	LangChain/OpenAI optional LLM grounded generation, prompt templates, deterministic offline fallback.
NLP/ML	PyTorch multi-task classifier, shared neural layers, category head, priority head, weighted cross-entropy, AdamW, dropout, early stopping, confidence thresholds.
Baseline ML	TF-IDF vectorization and Logistic Regression classifier for comparison against the PyTorch model.
Persistence	SQLite persistence plus JSONL logs for queries, feedback, metrics, and ticket drafts.
Security	API key authentication, JWT authentication, role-based access for admin, support agent, and viewer workflows.
Integrations	Mock or webhook adapters for ITSM ticket creation and chat/email notification workflows.
Evaluation	Evaluator script, curated test questions, retrieval metrics, groundedness, faithfulness, routing metrics, ML accuracy and F1.
Deployment	Docker, Docker Compose, GitHub Actions CI, Kubernetes manifests, ConfigMap, Secrets, Services, Ingress, PVC storage.

8. Interview Talk Track

Thirty-second summary:

I built an enterprise support automation platform where a user can ask an IT support question or upload documents. The backend uses LlamaIndex Workflows to orchestrate retrieval, grounded answer generation, ticket classification, priority prediction, routing, confidence scoring, and ticket draft creation. The RAG pipeline combines ChromaDB vector search, BM25 keyword search, metadata filtering, and reranking. I also added supervised NLP/ML with a PyTorch multi-task classifier and compared it against a Logistic Regression baseline.

Best points to emphasize:

- This is an intelligent system project, not only a chatbot.
- RAG is used to narrow the knowledge base and produce source-backed answers.
- Agent orchestration is used to decide whether to answer, ask clarification, escalate, or draft a ticket.
- NLP/ML is used for category and priority prediction, with measurable accuracy and F1 metrics.
- Production readiness is shown through auth, persistence, logging, Docker, Kubernetes, and CI.

One-line resume positioning:

Built an enterprise agentic RAG support automation platform using FastAPI, LlamaIndex Workflows, ChromaDB, BM25, Streamlit, and PyTorch to automate grounded IT support answering, ticket triage, priority prediction, routing, and ticket draft generation.